

First Try: Draft Algorithm

```
Traverse(id):
  • if exist id
    ◦ remove id
    ◦ str += id
    ◦ Traverse(next(id))
    ◦ str += tab per "/"
    ◦ Traverse(branch(id))
  • else str += "\n"
```

Go to the deepest possible location as it can. If nothing next, return "\n" and jump back to the previous ID, branch, and go to the deepest again.
This action is repeated recursively

which `next()` is get next to zettelkasten id: `akg5dx/a --> akg5dx/a1`
which `branch()` is get new branch id: `akg5dx/a --> akg5dx/b`

But str will contains empty line `\t\t\t...` when jump back & branch

Second Try: Use array instead str

Ignore (Filter) line has only `\t`: `array.filter(value => value.trim()).join("\n")`

- We recognize instead of using string, we can use array. Instead of `str+= "\n"`, just `push("")` - new item into array.
- Finally, we will filter array & merge (join) array with `"\n"` delimiter

So we have better version like:

```
function traverse(id,depth){
  let globalArr = []
  let maxDepth = 1
  const _next = (id) => {
    const prefix = id.slice(0,-1)
    const lastChar = id.slice(-1)
    if (/[a-zA-Z]/.test(lastChar)) return prefix + lastChar + "1"
    else return prefix + String(parseInt(lastChar) + 1)
  }
  const _branch = (id) => {
    const prefix = id.slice(0,-1)
    const lastChar = id.slice(-1)
    const charcode = lastChar.charCodeAt(0)
    if (/\\d/.test(lastChar)) return prefix + lastChar + "/a"
    return prefix + String.fromCharCode(charcode+1)
  }

  const _traverse = (id,depth) => {
    if (obj[id]){
```

```

        maxDepth = depth > maxDepth ? depth : maxDepth
        delete obj[id]
        if ( globalArr.length === 0 ) globalArr.push(id);
        else {
            globalArr[globalArr.length-1] = globalArr.at(-1) + id + "->"
        }
        _traverse(_next(id),depth+1)
        globalArr[globalArr.length-1] = globalArr.at(-1) + "\t".repeat(depth)
        _traverse(_branch(id),depth)
    }else {
        globalArr[globalArr.length-1] = globalArr.at(-1)
        globalArr.push("")
    }
}
_traverse(id,0)

```

Note 1: Passing depth for tracking which stack level is

Note 2: use `String.fromCharCode(charcode + 1)` to get next char in alphabet

Third Try: instead of `\t\t` to align, how about using `("".padEnd(num, " ")).repeat(depth)`

Figure out if we padEnd ID = n (10) & padEnd alignment = n + 4 (14). This will alignment beautifully

```

sczxy3/a → sczxy3/a1 → sczxy3/a2 → sczxy3/a3 → sczxy3/a4 → sczxy3/a5 →
sczxy3/a5/a → sczxy3/a5/a1 →
sczxy3/a5/a1/a →
sczxy3/a5/b →
sczxy3/a3/a →
sczxy3/b → sczxy3/b1 →
sczxy3/c →

```

But the problem is that if id is longer, it will break the rules

Fourth Try: Using table for alignment

Figure out, if we using table, it perfect for alignment. Return final string as below:

```

const str = `
| sczxy3/a → | sczxy3/a1 → | sczxy3/a2 → | sczxy3/a3 → | sczxy3/a4 → | sczxy3/a5 → | |
| | | | | sczxy3/a5/a → | sczxy3/a5/a1 → |
| | | | | | sczxy3/a5/a1/a → |
| | | | | | sczxy3/a5/b → | |
| | | | | sczxy3/a3/a → | | |
| sczxy3/b → | sczxy3/b1 → | | | |
| sczxy3/c → | | | | |
`

```

Better version:

```

...
const _traverse = (id, depth) => {
  if (obj[id]) {
    maxDepth = depth > maxDepth ? depth : maxDepth
    delete obj[id]
    if (globalArr.length === 0) globalArr.push(" | " + _expandIdIfRoot(id) + " →
");
    else {
      globalArr[globalArr.length - 1] = globalArr.at(-1) + " | " +
_expandIdIfRoot(id) + " → "
    }
    _traverse(_next(id), depth + 1)
    globalArr[globalArr.length - 1] = globalArr.at(-1) + " | ".repeat(depth)
    _traverse(_branch(id), depth)
  } else {
    globalArr[globalArr.length - 1] = globalArr.at(-1) + " | "
    globalArr.push("")
  }
}
_traverse(id, 0)
const str = `
${"|"}.repeat(maxDepth + 2)}
${"|---".repeat(maxDepth + 1) + "|"}
${convert(globalArr.filter(f => /[a-zA-Z]/.test(f)).map(line => line.slice(1) + " |
".repeat((maxDepth + 2 - (line.split("|").length - 1)))).join("\n"))}
`

```

```

${"|"}.repeat(maxDepth + 2)}
${"|---".repeat(maxDepth + 1) + "|"}
↳ Table Header Markdown

```

```

${convert(globalArr.filter(f => /[a-zA-Z]/.test(f)).map(line => line.slice(1) + " |
".repeat((maxDepth + 2 - (line.split("|").length - 1)))).join("\n"))}
↳ line.slice(1): cause line will start at "space", ignore it to correct Table Markdown
Syntax
↳ .repeat... : fill "|" so that match column in Table Markdown

```

Fifth Try: But we do not satisfy with current output

Instead of

```

sczxy3/a → sczxy3/a1 → sczxy3/a2 → sczxy3/a3 → sczxy3/a4 → sczxy3/a5 →
                                     sczxy3/a5/a → sczxy3/a5/a1 →
                                           sczxy3/a5/a1/a →
                                               sczxy3/a5/b →
                                                    sczxy3/a3/a →

sczxy3/b → sczxy3/b1 →
sczxy3/c →

```

We want to

```

sczxy3/a → sczxy3/a1 → sczxy3/a2 → sczxy3/a3 → sczxy3/a4 → sczxy3/a5 →
                                     sczxy3/a3/a →
                                     sczxy3/a5/a → sczxy3/a5/a1 →
                                     sczxy3/a5/b →
                                     sczxy3/a5/a1/a →

sczxy3/b → sczxy3/b1 →
sczxy3/c →

```

- Ideas to archive it as below:

1. Use regex to divide str:
 - Example: divide into 3 parts: [sczxy3/a...,sczxy3/b...,sczxyz/c....]
2. Each match will split into 2 parts: Header (first line) & Body (the rest)
3. In body, count "|" .." appears n time then append this line to Dict[n]
 - Note that: only count before | sczxy3...
 - So, we obtains Dict like:

```

{
  '0': '',
  '3': '| | | | | sczxy3/a3/a → | | | | ',
  '5': '| | | | | | sczxy3/a5/a → | sczxy3/a5/a1 → |\n' +
    '| | | | | | sczxy3/a5/b → | | ',
  '6': '| | | | | | | sczxy3/a5/a1/a → | '
}

```

4. Loop through keys in Dict, generate newBody
 - Note that: keys in object default sort by ascending
5. Return New String Converted

```

function convert(str) {
  const regex = /((?<=^|).+?(?=(^| [a-zA-Z]))|(?![\s\S]))/gms;
  const matches = str.match(regex);
  let result = ""

  for (let match of matches) {
    const dictDepthToLine = {}; // "depth_level": "line str"
    const lines = match.split("\n");
    const header = lines[0];
    const body = lines.slice(1);
    // | | | | | | sczxy3/a5/a → | sczxy3/a5/a1 →
    // count how many "|" " before id
    const lineRegex = /\|\\s(?=[^a-zA-Z])/g;
    body.forEach((line) => {
      let depthLevel = 0;
      if (line.includes("→")) {
        depthLevel = line.slice(0, line.indexOf("→")).match(lineRegex)?.length ??
0;
      }
      dictDepthToLine[depthLevel] = !dictDepthToLine[depthLevel]
        ? line
        : dictDepthToLine[depthLevel] + "\n" + line;
    });
    const newBody = Object.keys(dictDepthToLine)
      .map((key) => dictDepthToLine[key])
      .join("\n");
  }
}

```

```

        console.log(dictDepthToLine)
        result += "|" + header + newBody + "\n"
        // console.log(match.split("\n"))
    }
    return result
}

```

Sixth Try: Try to make width td fixed content

```

table {
    max-width: fit-content !important
}

```

Seventh Try: Table with no-border

```

table td, table th, table {
    border: none !important;
    border-collapse: collapse !important;
    background-color: transparent !important; /* Ensures no background shadows */
    box-shadow: none !important
}
table th {
    display: none
}

```

Eigth Try: Make Table Look Cleaner

```

table td{
    overflow: hidden;
    text-overflow: ellipsis; /* Adds "..." if text is too long */
    white-space: nowrap;      /* Prevents wrapping (remove this if you want wrapping) */
}

```

Nineth Try: Trick for assign Markdown Attribute for Table (OR Others)

- So far, we use `Markdown Attribute` for assign class to table,ul,p,div,...
- But we can not use `Markdown Attribute` inside `meta-bind-js-view`
- Here is a trick to archive that: 1. Create new markdown file (newFile.md) with display things we want to style. (In this example, this is meta-bind-js-view with display Table Markdown) 2. Instead display things we want to style directly, just embed this newFile.md such as: `[[newFile.md| custom-attribute]]` 3. In CSS Snippets, things we want to style will inside `<span alt="custom-attribute"...</span>` 4. Just style with that selector:

```

span.internal-embed[alt*="traverse-zettelkasten-id"] table{
  max-width: fit-content !important;
}

span.internal-embed[alt*="traverse-zettelkasten-id"] table td{
  overflow: hidden;
  text-overflow: ellipsis; /* Adds "..." if text is too long */
  white-space: nowrap; /* Prevents wrapping (remove this if you want wrapping) */
}

/* 1. Remove borders from the table, rows, and cells */
span.internal-embed[alt*="traverse-zettelkasten-id"] table,
span.internal-embed[alt*="traverse-zettelkasten-id"] table th,
span.internal-embed[alt*="traverse-zettelkasten-id"] table td,
.markdown-source-view.mod-cm6 .cm-content table,
.markdown-source-view.mod-cm6 .cm-content table th,
.markdown-source-view.mod-cm6 .cm-content table td {
  border: none !important;
  border-collapse: collapse !important;
  background-color: transparent !important; /* Ensures no background shadows */
}

/* 2. Remove the table's outer border (if any) */
span.internal-embed[alt*="traverse-zettelkasten-id"] table {
  border: none !important;
  box-shadow: none !important; /* Removes potential shadow borders */
}

span.internal-embed[alt*="traverse-zettelkasten-id"] table th{
  display: none;
}

```